C200 Programming Assignment № 1

Functions & Learning About Homework
Fall 2023

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

January 19, 2024

The purpose of this homework is two-fold: (1) you'll develop your skills in implementing functions, formal and actual parameters, environment and scope (2) how homework will be delivered and submitted. Although the problems can be used for individuals, this course uses paired-programming. Please carefully read about pair-programming below.

Instructions

The course uses an online system for grading the Python code. The system can be accessed at `c200.luddy.indiana.edu`, and you should login via your official IU username and password. Upon login, you can submit HW via 'Create Submission' button. Remember that you **must** push the HW files to GitHub and also submit to the Autograder before the HW deadline. We open submissions option in the Autograder a few days after the HW is released. If you don't see the relevant Assignment tab in the Autograder, please feel free to check back peridiocally.

Paired-Programming

Homework is generally done with a randomly selected partner. In the last section, some information about contacting and communicating with this likely new student will be facilitated. **As always, all the work will be with only you and your partner; but *both* of you should contribute.** If your partner does not respond, you must complete the homework on your own. You will not be assigned a new partner–this is infeasible for such a large class. Both students must submit their individual homework, since you are graded individually. Your grade may be different from your partner's grade, since there can be differences in code.

You must complete this before **Friday, January 26, 2024 11:00 PM EST**. You will submit your work by pushing your code to your GitHub repository and by uploading your completed python file to the Autograder. Please remember that.

- You will **not** turn anything in on canvas.

- You do **not manually upload files** to your repository using GitHub's "Upload files" tool.
- You must submit your individual program even though you have a partner, since you'll be graded individually.
- Your github will accept pushes at *any* time. Once the deadline for homework is passed, if you push again, you'll rewrite over the correct time and the solution will be past the deadline and not graded. If your timestamp is 11:01PM or later, the homework will not be graded under almost any circumstance. So do not wait until 10:59PM to commit and push your changes. Pushing with only a few minutes left is **not** a valid reason to be graded.
- You cannot discuss homework outside of your partner, use tools like Chat-GPT, copy a solution–a person who cheats cheats himself.
- Struggling is part of any practiced skill–it's how we learn.

Remarks: (1) We do not print or return units–only values. If a function is calculating a percentage, dollar amount, *etc.* the percentage sign % is **not** shown, the dollar sign $ is not shown. For the problem descriptions, we occassionally include them to make interpreting the problem easier. (2) Often numerical values will be infinite. Python will give you **by default** about 15 decimal places. We'll learn how to shorten these values later. For now, for example we'll write 104.17 for 104.16666666666667. On this HW, you are allowed to use round() function to round the final answer before returning it in the funtion-for example, if your final answer is 6.85457 then it's rounded equivalent is 6.85, and can be obtained by doing **round(6.8547, 2)**. The first parameter to the round() function is the actual value that you want to round, and the second argument is the number of places you want it rounded to. Sometimes, we don't want to round–we simply want the integer portion. Recalling from lecture we can write **int(·)**. For example, int(1.5) = 1.

Comments about Homework

You will be given the function signature without a body. For each problem, you'll replace the Python keyword **pass** with code. Do not change any signature. If you do, the autograder will fail and you'll receive a zero. If you add any extra functions, they must be local to the function you are building; otherwise, the autograder will fail. This is a very short homework to help you get started with understanding the process.

Keyboard Quickies

To uncomment multiple lines (Windows):

1. select the lines to be commented
2. Ctrl+k+u

To comment multiple lines (Windows):

1. select the lines to be commented
2. Ctrl+/

To comment or uncomment multiple lines (Mac):

1. select the lines to be commented
2. cmd + /

To undo (Windows):

- Ctrl+Z

To undo (Mac)

- Command+Z

Some of these problems were taken or inspired by the excellent introductory texts *Applied Calculus* by Tan, 2005, *Thinking Mathematically* 5th *ed.* by Blitzer, 2011.

Problem 1: Volume of a Cone

The volume of a cone with radius r and height h is:

$$c(r,h) = \frac{1}{3}\pi r^2 h \tag{1}$$

For example, 2 cm radius and 5 cm height,

$$c(2,5) \approx 20.94\ cm^3 \tag{2}$$

For 3 cm radius and 7 cm height

$$c(3,7) \approx 65.97\ cm^3 \tag{3}$$

Deliverables Problem 1

- Complete the cone volume function in the file a1.py.
- You must use math.pi from the math module.
- Round the answer to 2 decimal places. Use round(x,y) to round x to y decimal places. For example,

```
>>> round(1.252,2)  #rounds to two places
1.25
>>> round(1.252,1)  #rounds to 1 place
1.3
>>> round(1.252,0)  #rounds to 0 place
1.0
```

As an example, here's code with rounding two 1 place in the function:

```
def foo(x,y,z):
    ans_ = round((x + y + z)/3,1)
    return ans_

if __name__ == "__main__":

    """
    If you want to do some of your own testing in this file,
    please put any print statements you want to try in
    this if statement.

    You **do not** have to put anything here  """

    # my test
    print(foo(200,300,500))
```

has output:

```
333.3
```

Problem 2: Oxygen Content of a Pond

The oxygen content t days after organic waste has been dumped into a pond is given by:

$$f(t) = 100\frac{t^2 + 10t + 100}{t^2 + 20t + 100} \tag{4}$$

percent of its normal level. For example,

$$f(0) = 100 \tag{5}$$

$$f(10) = 75 \tag{6}$$

Deliverables Problem 2

- Complete the oxygen content function in the file a1.py.
- Round the value you function returns to 2 decimal places.

Problem 3: TV Viewing Patterns

According to A.C. Nielsen Co., the percent of U.S. households waching television during the weekdays (about a decade ago) starting at 4:00PM for eight hours is modeled as $P(t)$:

$$P(t) = 0.01354t^4 - 0.49375t^3 + 2.58333t^2 + 3.8t + 31.60704 \tag{7}$$

if $0 \leq t \leq 8$ where $t = 0$ corresponds to 4:00P. For example,

$$P(0) = 31.61\% \tag{8}$$

$$P(3) = 54.02\% \tag{9}$$

$$P(8) = 30.0\% \tag{10}$$

Deliverables Problem 3

- Complete tv percent function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 4: Toxic Waste

A city's main well was recently found to be contaminated with trichloroethylene, a cancer-causing chemical, as a result of an abandoned chemical dump leaching chemicals into

the water. A proposal submitted to city council members indicates that the cost, measured in millions of dollars to remove x% of the toxic pollutant is given by:

$$cost(x) = \frac{0.5x}{100 - x} \tag{11}$$

for $0 < x < 100$. For eample, 50%, 70%, and 90% cost

$$cost(50) = \$.5 \text{ million} \tag{12}$$

$$cost(70) \approx \$1.17 \text{ million} \tag{13}$$

$$cost(90) = \$4.5 \text{ million} \tag{14}$$

Deliverables Problem 4

- Complete the percent cost function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 5: Cowling's Rule

Cowling's rule is a method for calculating pediatric drug dosages. If a denotes the adult dosage(in milligrams) and t is the age of the child (in years), then the child's dosage is given by:

$$D(t, a) = \frac{t+1}{24}a \tag{15}$$

For example, if $a = 500 \text{ mg}$ and $t = 4 \text{ years}$, then

$$D(t, a) = 104.17 \text{ mg} \tag{16}$$

$$\tag{17}$$

Deliverables Problem 5

- Complete Cowling's rule function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 6: Flu Outbreak

During a flu outbreak in a school of 1000 children, the number of infected children, I, was expressed in terms of the number of susceptible (but still healthy) children, S, by:

$$I(S) = \lceil 192 \log_2(\frac{S}{762}) - S + 763 \rceil \tag{18}$$

For example,

$$I(100) = 101 \text{ students} \tag{19}$$

$$I(300) = 205 \text{ students} \tag{20}$$

Deliverables Problem 6

- The math module is imported for this problem.
- Complete the I function in the file a1.py.
- You may get negative values for some inputs values that you provide to the function, and that's fine.
- To remind you, $\lceil x \rceil = y$ is the smallest integer y such that $x \leq y$.
- Use **math.ceil()** from the math module discussed in lecture.

Problem 7: Average Cost

Let $C(q)$ be the cost of producing q items. A company's cost function is (in $) given by

$$C(q) = 0.01q^3 - 0.6q^2 + 13q + 1000 \tag{21}$$

The **average cost** $A(q)$ is given by

$$A(q) = \frac{C(q)}{q} \tag{22}$$

Deliverables Problem 7

- Complete both functions C, A in the file a1.py
- For this problem, you'll need to make your own input/outputs

Problem 8: Sales Model

A company contracted by you has built a sales model that returns the number of plastic ducks sold ($\times 10^3$)

$$hh(t) = \left\lfloor \frac{532}{1 + 869e^{-1.33t}} \right\rfloor \tag{23}$$

for $0 \leq t \leq 11$ where t is months. The floor function $\lfloor\ \rfloor$ is in the math module named math.floor() For example,

$$hh(0) = 0 \text{ ducks} \tag{24}$$

$$hh(5) = 25 \text{ ducks} \tag{25}$$

$$hh(10) = 531 \text{ ducks} \tag{26}$$

Deliverables Problem 8

- The math module is already imported in a1.py.
- Complete the hh function in the file a1.py.
- Round your answer to the nearest integer using math.floor()

Problem 9: Throwing a Stone

A stone is thrown straight up from the roof of an 80 ft building with an initial velocity of $64\,\mathrm{ft/sec}$. The height (in feet) of the stone at any time t seconds is given by:

$$height(t) = -16t^2 + 64t + 80 \tag{27}$$

The rock will hit the ground after 5 seconds:

$$height(5) = 0 \text{ feet} \tag{28}$$

Deliverables Problem 9

- Complete the $height$ function in the file a1.py
- Round the answer to 2 decimal places.

Problem 10: Treating Heart Attacks

According to the American Heart Association, the treatment benefit for heart attacks depends on the time (hours) until treatment and is described by:

$$B(t) = \frac{0.44t^4 + 700}{0.1t^4 + 7} \tag{29}$$

for $(0 \leq t \leq 24)$

Deliverables Problem 10

- Complete the B function in the file a1.py.
- For this problem, you'll need to make your own input/outputs.
- Round the answer to 2 decimal places.

Problem 11: Roots to the Quadratic

Recall that a quadratic is a function:

$$q(x) = ax^2 + bx + c \tag{30}$$

A root is a number that makes the function zero. For example, if

$$q(x) = 2x^2 + 5x - 12 \tag{31}$$

then the two roots are $x = -4$ and $x = 3/2$:

$$q(-4) = 2(-4)^2 + 5(-4) - 12 = 32 - 20 - 12 = 0 \tag{32}$$

$$q(3/2) = 2(3/2)^2 + 5(3/2) - 12 = 2(9/4) + 15/2 - 12 = (24/2) - 12 = 0 \tag{33}$$

Implement a function quad(a,b,c,x) that returns True if x is a root for $ax^2 + bx + c$ and False otherwise. Running the function:

```
print(quad(2,5,-12,-4))
print(quad(2,5,-12,3/2))
print(quad(2,5,-12,1))
```

gives output:

```
True
True
False
```

Deliverables Problem 11

- Complete the $quad$ function in the file a1.py

Problem 12: Sinking Fund

This model describes a “sinking fund." Money is periodically set aside until a date is reached. One important kind is retirement–money that will be available when you stop working. You must know the payment amount P, the number of times a year you make the payment n, the number of years you make the payment t, and the interest rate r. For example,

- If you pay $22,000 once a year for seven years that has 6% compounded annually, you’ll have, at the end $184,664.43.
- If you make $500 monthly that has 4% (about the current best rate) compounded monthly for 20 years, you’ll have $183,387.31
- If $1,200 is deposited quarterly with 8% compounded quarterly for 10 years, you’ll have $72,482.38

The sinking fund is:

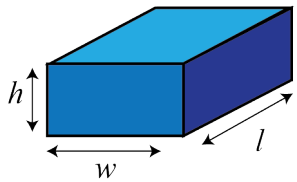
$$R(P, r, n, t) \quad = \quad P\left[\frac{(1+\frac{r}{n})^{nt}-1}{\frac{r}{n}}\right] \tag{34}$$

Deliverables Problem 12

- Complete the R function in the file a1.py
- Round to two decimal places.

Problem 13: Rectangular Solid

Complete the Python function that gives the total surface area S of a rectangular solid with dimensions w, l, h shown here:



The surface area is:

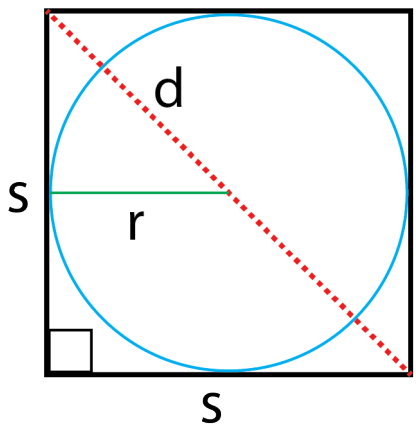
$$S(w, l, h) \quad = \quad (?) \tag{35}$$

Deliverables Problem 13

- Complete the S function in the file a1.py
- You must create your own solutions for this problem.

Problem 14: Geometry

You are given two functions to implement. The first takes the length of a side of a square and returns the diagonal (no units). The second takes a diagonal of a square and returns the area of the largest circle that can be inscribed in the square. A figure is shown below.



The side is s, the diagonal (red dotted line) is d, the radius (green) is r, and the circle blue. The easiest solution involves using the Pythagorean Theorem. Here is a run:

```
s = 10
a  = circle_area(square_diagonal(s))
print(a)
```

has output:

```
78.54
```

where we have rounded to two decimal places. As you can see, there is function composition: the output of the first function is the input to the second.

Deliverables Problem 14

- Complete the two functions for this problem in the file a1.py
- You can create some of your own tests to verify your implelmentation is correct.
- Round to two decimal places.

Problem 15: Toward Sports Analytics

A pitcher's earned-run average written ERA is determined by:

$$\text{ERA}(e, i, t) \quad = \quad (\tfrac{e}{i})t \tag{36}$$

where e is earned runs, i is innings pitched, and t is total innings. For example, if a pitcher had 4 earned runs, pitched 6 innings, and the total number of innings was 9, then the ERA is:

$$\text{ERA}(e, i, t) \quad = \quad (\frac{4}{6})9 = 6 \tag{37}$$

Deliverables Problem 15

- Complete the ERA function in file a1.py
- You can create some more test yourself to verify your implelmentation is correct.
- Round to two decimal places.

Problem 16: Climate

Wind chill is a somewhat vague phenomenon associated with the perception of temperature drop that a person feels. There does not exist any standard for wind chill, but in the U.S., the wind chill index T_{wc} is:

$$T_{\text{wc}} = 35.74 + 0.6215T_a - 35.75v^{0.16} + 0.4275T_a v^{0.16} \tag{38}$$

For example, if the temperature is $2\,°$C and wind velocity is $5\,$mph, then the wind chill is $-9\,°$C.

Deliverables Problem 16

- Complete the T_wc function in file a1.py
- Create some of your own tests to verify your implelmentation is correct.
- Use the math floor function on the value.

Problem 17: Guessing (Approximation)

Most of the time we cannot easily calculate a value, *e.g.*, too big, too small, too complicated. In computing we have learned to guess (or what we call *approximate*). You're no doubt familiar with factorial $n!$, but did you know it was known in the 18th century? Without computers, calculating $n!$ for even what we would consider tiny values was quite difficult. An eponymous approximation, called the Stirling approximation, is (as a lower bound):

$$n! \approx \sqrt{2\pi n}(\frac{n}{e})^n \tag{39}$$

While this might actually look more difficult it isn't. The following code:

```
n0 = 10
print(math.factorial(n0),fact_est(n0))
n0 = n0 * 10
print(math.factorial(n0),fact_est(n0))
```

produces:

```
3628800 3598695

93326215443944152681699238856266700490715968264381621
468592963895217599993229915608941463976156518286253697920827223758251185210916864000000000000000000000000000
```

```
932484762526941958317139521732058634340918920550962 63
580576295198856440963432534655500101747343711504865 77
945489842010123445711323002886979033757847852469452 8
```

When you run this, it'll look different–because the numbers have so many digits, the values don't easily lend themselves to print.

Deliverables Problem 17

- Complete the function in file a1.py
- Please create some additional test to verify your implelmentation is correct.
- Return the approximation as an integer (cast it as an int).

Communication

When connecting with a new person, you should introduce yourself, give a short message describing the purpose, then salutation. Please use and adapt this format when connecting to a new partner. The student pairs are given on the next page.

```
Dear Student,

My name is X Y and I'm a student in your C200,
but a different section.  It looks like we're
partners this week.  I'm generally free on
Thursday, Saturday, Sunday.  I'm hoping we
can knock most of this out on our first meeting.
Luddy is a great place if we can snag a
conference room.

Take care,

Another Student
```

Format for InScribe post

```
Hi Instructor/ TA <Whom ever they want to
ask question/ Share something>,

If you are posting about lecture/Notes
My name is Student-A. I am in Dr. (mention faculty instructor name)
lecture section. This is regarding ....

If you are posting about labs
My name is Student-A. I am in the lab section that meets on
DAY, TIME of your lab and NAME OF THE TA.
This is regarding ....

Thanks and regards,
<Name>
<User name>
```

Programming partners

josajack@iu.edu, jamread@iu.edu
sjbarry@iu.edu, aquintas@iu.edu
cedonica@iu.edu, gravipat@iu.edu
jaehwan@iu.edu, msolomie@iu.edu
mgrass@iu.edu, chsabir@iu.edu
nsfried@iu.edu, katepate@iu.edu
nkushnir@iu.edu, ymarrakc@iu.edu
craftdc@iu.edu, bsai@iu.edu
hedavid@iu.edu, gmwerlin@iu.edu
jamlian@iu.edu, armnguy@iu.edu
ykassahu@iu.edu, shubtiwa@iu.edu
zcarmich@iu.edu, wardjohn@iu.edu
andhodge@iu.edu, adpulipa@iu.edu
boding@iu.edu, jlopezmo@iu.edu
changali@iu.edu, saravula@iu.edu
aydclink@iu.edu, camlueck@iu.edu
conkeen@iu.edu, johnsanc@iu.edu
ryandaun@iu.edu, srimacha@iu.edu
jakedeck@iu.edu, smremmer@iu.edu
josthill@iu.edu, wangjinz@iu.edu
samjimen@iu.edu, ceprenti@iu.edu
lrborows@iu.edu, ksreejit@iu.edu
blevidut@iu.edu, zstopher@iu.edu
jailjohn@iu.edu, snlnu@iu.edu
kimyouh@iu.edu, archshar@iu.edu
gmhowell@iu.edu, avaweish@iu.edu
abelani@iu.edu, smunikun@iu.edu
cfahy@iu.edu, knrumsey@iu.edu
adeglur@iu.edu, tryerrag@iu.edu
mbadwe@iu.edu, fshamrin@iu.edu
langgrav@iu.edu, arangi@iu.edu
chenkay@iu.edu, nmetlapa@iu.edu
tkalale@iu.edu, twirges@iu.edu
ejbusch@iu.edu, brshelby@iu.edu
spebrook@iu.edu, wagnebr@iu.edu
malhamad@iu.edu, ashahi@iu.edu
ecabebe@iu.edu, rahkpate@iu.edu
zlangley@iu.edu, benserna@iu.edu

kespejo@iu.edu, abbalokh@iu.edu
blberger@iu.edu, auspiege@iu.edu
ml82@iu.edu, mxrobins@iu.edu
chodgman@iu.edu, ctsvetan@iu.edu
logellis@iu.edu, breramer@iu.edu
oluanifo@iu.edu, sskiadop@iu.edu
cc96@iu.edu, weinstab@iu.edu
agunde@iu.edu, shshar@iu.edu
harrisdg@iu.edu, amtsmith@iu.edu
jabbarke@iu.edu, etmillho@iu.edu
adevill@iu.edu, frvisco@iu.edu
tkonkim@iu.edu, mitcchar@iu.edu
amgandhi@iu.edu, mvsimi@iu.edu
burkhel@iu.edu, pratmodi@iu.edu
kylrguth@iu.edu, nvoneper@iu.edu
jdemian@iu.edu, masoroki@iu.edu
allentp@iu.edu, ishetty@iu.edu
auconner@iu.edu, arstover@iu.edu
kjj6@iu.edu, shmulla@iu.edu
rcerauli@iu.edu, nicmill@iu.edu
akumares@iu.edu, amoza@iu.edu
marbelli@iu.edu, alevalti@iu.edu
pkurzawa@iu.edu, ptapiaro@iu.edu
smhamrah@iu.edu, erugova@iu.edu
reafrank@iu.edu, rtripat@iu.edu
edjgarci@iu.edu, javada@iu.edu
shbivens@iu.edu, joebrahm@iu.edu
kayfaris@iu.edu, iopfings@iu.edu
njustin@iu.edu, cmoverfe@iu.edu
sebcont@iu.edu, sparab@iu.edu
bcabral@iu.edu, manushah@iu.edu
daveanu@iu.edu, ap59@iu.edu
belleli@iu.edu, leorusyn@iu.edu
camehugh@iu.edu, tmukund@iu.edu
shaagore@iu.edu, syeshala@iu.edu
anchinna@iu.edu, teswank@iu.edu
rkeel@iu.edu, javitto@iu.edu
presharr@iu.edu, mmaleck@iu.edu
nabatja@iu.edu, traepena@iu.edu
nbkotur@iu.edu, cmm23@iu.edu

benitezb@iu.edu, joprayit@iu.edu
cjduffie@iu.edu, haoztian@iu.edu
beckab@iu.edu, mackeyr@iu.edu
jhbruce@iu.edu, surehma@iu.edu
pafrost@iu.edu, vupatel@iu.edu, tiffyou@iu.edu
sdharmmi@iu.edu, camyee@iu.edu
emiellis@iu.edu, trimedio@iu.edu
ankotel@iu.edu, alexpan@iu.edu
sgudapur@iu.edu, msantell@iu.edu
wbrogan@iu.edu, colbstap@iu.edu
nkhai@iu.edu, martichc@iu.edu
sambereb@iu.edu, bpovinel@iu.edu
lineri@iu.edu, peterant@iu.edu
lasdodd@iu.edu, andmuchi@iu.edu
cdinman@iu.edu, peterabl@iu.edu
allennam@iu.edu, carlmont@iu.edu
markdeng@iu.edu, bwmoore@iu.edu
jaxkatz@iu.edu, lomwean@iu.edu
lawlin@iu.edu, jocpoiri@iu.edu
zaycarey@iu.edu, olipfing@iu.edu
reychaud@iu.edu, rimaeda@iu.edu
berrynm@iu.edu, crschuit@iu.edu
careyjo@iu.edu, reddysk@iu.edu
askaul@iu.edu, kammich@iu.edu
anudivek@iu.edu, wmccarte@iu.edu
jadebail@iu.edu, khwillar@iu.edu
madimond@iu.edu, leoett@iu.edu
renahan@iu.edu, animerch@iu.edu
jvgottum@iu.edu, ntorpoco@iu.edu
forlin@iu.edu, austrahl@iu.edu
jkchaura@iu.edu, amatariy@iu.edu
eynkansa@iu.edu, menonvi@iu.edu
rachguo@iu.edu, stewti@iu.edu
lbadredd@iu.edu, jianlou@iu.edu
cgreywal@iu.edu, corrosar@iu.edu
eberens@iu.edu, gstogdil@iu.edu
isocasey@iu.edu, bfmark@iu.edu
ocrowder@iu.edu, nsenthi@iu.edu
ecanant@iu.edu, jomalek@iu.edu
cbremner@iu.edu, jmatczak@iu.edu

aadkhan@iu.edu, rsreedh@iu.edu
balian@iu.edu, ggriorda@iu.edu
harmonnc@iu.edu, jusshah@iu.edu
arkhal@iu.edu, mgstack@iu.edu
chamblia@iu.edu, nrimawi@iu.edu
achitred@iu.edu, xsoto@iu.edu
arkali@iu.edu, nsmurthy@iu.edu
wchorich@iu.edu, njvinson@iu.edu